

Università
degli Studi
di Padova

University of Padua

Department of Mathematics “Tullio Levi-Civita”

Formal Methods for Cyber-Physical Systems

ASSIGNMENT 1

Luca Monetti

Part 1

Assignment

The first part of the assignement request the implementation of the symbolic algorithm fo invariant verification. If the test property is not an invariant of the model the algorithm also provide an execution of the system that violate the invariant.

In particular the algorithm must respect the following specification:

- Check if `spec` is an invariant for the loaded model
- The return value is a `tuple` of 2 elemens:
  - The first element is a Boolean value that is `True` if the formula is an invariant, `False` otherwise
  - The second element is `None` is the formula is an invariant and contains an execution that violate that specification otherwise.
- The execution take the form of a tuple of alternating states and input, ending with a state.

Logic

Implementation

Part 2

Assignment

The second part of the assignment request the implementation of a symbolic algoritm for the verification of a specific class of LTL formulas called *response formulas*. They have a specific form:

$$\square(f \rightarrow \diamondsuit g)$$

Both f and g are Boolean combinations of base formulas with no temporal operators.

In the following pages will be described an implementation of the function `check_examplain_response_spec(spec)` that respect the following specification:

- Check if the given `spec` is satisfied by the loaded SMV model
- The return value is a `tuple` of 2 elemens:
  - The first element is a Boolean value that is `True` if the formula is satisfied, `False` otherwise
  - The second element is `None` is the formula is satisfied but contains a counter-example otherwise.
- The counter-example take the form of a tuple of alternating states and input, ending with a state.
- If the execution is looping the last state should be present somewhere else in the sequence.

Logic

We are dealing with a *response formula* that we can explain such as:

Every time that f happens, g must eventually happen at the same time or afterwards.

Being a reactive property, and therefore happens over an infinite computation, to prove that the formula is not satisfied we must provide a counter-example in witch the negation is true. A path in witch f holds but g never happens again. For that we need 2 parts:

1. **Prefix**: A sequence of states leading to f
2. **Loop**: An infinite cycle in witch g is always `false`.

Implementation

The check of the formula structure was already done by the function `parse(spec)`, so we can just use it and check if the result is a valid `tuple` of f and g or `None`. In the second case the program shuld return `None`, beacause the formula is not a *response formula*.

After that, using the provided function `spec_to_bdd(model, spec)`, we can obtain the `bdd` generated from the two needed formulas: f and $\neg g$.

The algorithm will operate only on the reachable states, so we get them using `model.reachable_states`. From this we can compute the variable `recur` that will contain all the possible starting point for a cycle. In our case all the reachable states in which f holds and $\neg g$ holds.

There are two possibile execution:

1. `recur` becomes empty, there are no possible cycle, the formula holds and the algorithm return a positive result.
2. Otherwise each iteration compute the `pre_reach`, all the state that can be reach `recur` in a finite number of steps greater than 0, and `new`, all the states that might be part of the cycle.

If a cycle is found then the algorithm will build a counter-example using the function `compute_counter_example(model, recur, pre_reach)`, which will be describe in the next section.

The loop runs until there are no more possibile candidates for a cycle.

```
def check_explain_response_spec(spec):
    """
    Return whether the loaded SMV model satisfies or not the GR(1) formula
    `spec`, that is, whether all executions of the model satisfies `spec`
    or not.
    """

    model = pynusmv.glob.prop_database().master.bddFsm
    react = parse(spec)
    if (react == None):
        return None

    f_spec, g_spec = react

    f_bdd = spec_to_bdd(model, f_spec)
    neg_g_bdd = spec_to_bdd(model, ~g_spec)

    reach = model.reachable_states
    recur = reach * neg_g_bdd * f_bdd

    while recur.isnot_false():
        pre_reach = pynusmv.dd.BDD.false()
        new = model.pre(recur) * neg_g_bdd

        while new.isnot_false():
            pre_reach = pre_reach + new
            if recur <= pre_reach:
                return (False, compute_counter_example(model, recur, pre_reach))
            new = (model.pre(new) - pre_reach) * neg_g_bdd
        recur = recur * pre_reach
    return (True, None)
```

Counter example generation

As said before a counter-example is an execution of the system that leads to a state for wich the formula is not satisfied. It is composed of 2 parts:ù

1. **Prefix**: A sequence of states leading to f
2. **Loop**: An infinite cycle in witch g is always `false`.

Those two parts are respectively computed by `compute_prefix` and `compute_loop`.

Build the path

The first computation is easy, we just need to compute a path that reach the starting node of the cycle. To do that, we start from the `model.init` set of states and we proced with the execution until we reach a `frontier` that contains that specific state.

After that we can construct the needed `tuple` visiting the array from the end to the beginning.

```
1  def compute_prefix(model, s):
2    has_inputs = len(model.bddEnc.inputsVars) > 0
3    # 2.3 Reaching the self loop
4
5    # Insert inside frontiers the regions to reach s
6    # [ init ... pre_s ]
7    reached = model.init
8    frontiers = [reached]
9
10   while (reached * s).is_false():
11       next = model.post(reached) - reached
12       frontiers.append(next)
13       reached = reached + next
14
15   path = []
16   last = s
17   # Construct the path traversing the frontiers in reverse
18   for frontier in reversed(frontiers[:-1]):
19       can_reach_last = model.pre(last)
20
21       # The current state is one of the states that can reach last in
22       # one step
23       current_state = model.pick_one_state(frontier * can_reach_last)
24
25       if has_inputs:
26           inputs = model.get_inputs_between_states(current_state, last)
27           path.insert(0, model.pick_one_inputs(inputs).get_str_values())
28       else:
29           path.insert(0, {})
30
31       path.insert(0, current_state.get_str_values())
32
33       last = current_state
34
```

```
35    return path
```

Build the loop

The loop is more complex. We need to compute a loop starting from `s`, following the path through the frontiers list.

We first need to identify the smallest number of steps `k` that contains `s`. We then define 2 invariant:

- `path` -> All the possible path ending in `s` in `k` steps.
- `current` -> Last state put in the `path`.

The algoritm build the counter-example iterating backwards the `frontiers` taking one state and its corresponding input at the time.

After the last iteration `path` contains a full path ending in `s` that starts from the first `frontier` region.

Unite those 2 path togheter leads to a path starting from an initial state, reaching a state in witch f holds and $\neg g$ holds, and lead to a cycle in witch $\neg g$ holds.

We can then be sure that the counter-example is built on the right way and rapresent a valid execution of the system that don't satisfies the formula.

```
1  def compute_loop(model, s, frontiers):
2    has_inputs = len(model.bddEnc.inputsVars) > 0
3
4    # Phase 2.2: Building the self loop
5    for k in range(len(frontiers)):
6        if s <= frontiers[k]:
7            break
8    # frontiers[k] contains the loop knot
9
10   path = [ s.get_str_values() ]
11   curr = s
12
13   for i in range(k - 1, -1, -1):
14
15       old = curr
16       pred = model.pre(curr) * frontiers[i]
17       curr = model.pick_one_state(pred)
18
19       if has_inputs:
20           inputs = model.get_inputs_between_states(old, curr)
21           path.insert(0, model.pick_one_inputs(inputs).get_str_values())
22       else:
```

```
            path.insert(0, {})

        path.insert(0, curr.get_str_values())

    # Looping input
    if has_inputs:
        inputs = model.get_inputs_between_states(s, curr)
        path.insert(0, model.pick_one_inputs(inputs).get_str_values())
    else:
        path.insert(0, {})

    path.insert(0, s.get_str_values())
    return path
```